

Deploying Metabase on AWS ECS (Fargate)

by Mohamad Ikhsan Nurulloh



Metabase



AWS Fargate

Disclaimer

- **No Endorsement:** This guide is for educational purposes only and does not imply endorsement by Metabase or AWS.
- **Testing Only:** Intended for testing and development, not production environments.
- **Refer to Official Docs:** Always consult official Metabase and AWS documentation for production use.
- **Proceed at Your Own Risk:** Use this guide at your own risk and be aware of potential deployment implications.

What is AWS ECS (Fargate)?



AWS Fargate

AWS Fargate is a technology that you can use with Amazon ECS to run [containers](#) without having to manage servers or clusters of Amazon EC2 instances. With AWS Fargate, you no longer have to provision, configure, or scale clusters of virtual machines to run containers. This removes the need to choose server types, decide when to scale your clusters, or optimize cluster packing.

Business Case

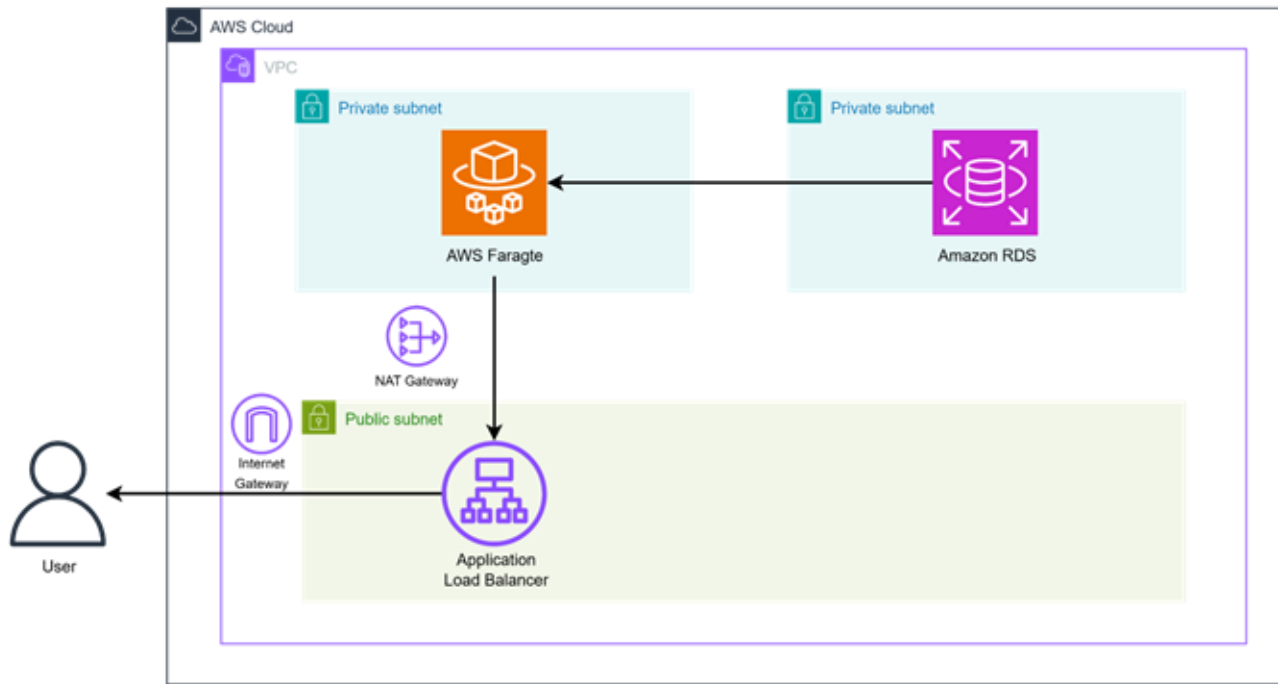


AWS Fargate

Deploying Metabase or similar applications on Amazon ECS Fargate using the AWS free tier provides a cost-efficient way to run containerized workloads without managing servers. This setup eliminates the need for EC2 instances and load balancers, simplifying infrastructure while keeping costs low. It is ideal for small to medium-sized deployments or development environments where budget constraints are important. This approach allows smooth scaling within free tier limits, reduces operational overhead, and speeds up deployment, making it a practical solution for running business intelligence applications like Metabase on a tight budget.

Metabase Architecture

Architecture



Overall Architecture: Metabase runs on AWS Fargate behind an Application Load Balancer (ALB) in a public subnet, which handles HTTPS traffic from users. The Metabase containers in private subnets store metadata in Amazon RDS, while a NAT Gateway enables safe outbound access. Security groups restrict traffic so only the ALB can reach Fargate and only Fargate can access RDS, delivering a secure, scalable, and fully managed analytics platform.

Details Metabase Architecture

- **VPC with Public & Private Subnets:** Public subnet hosts the ALB; private subnets host Fargate tasks and Amazon RDS for isolation and high availability.
- **Application Load Balancer (ALB):** Handles HTTPS from users via the Internet Gateway and securely routes requests to Fargate tasks.
- **AWS Fargate (ECS Service):** Runs Metabase containers without server management, scaling automatically to match demand.
- **Amazon RDS:** Provides a managed PostgreSQL/MySQL database in private subnets for storing all Metabase metadata and configuration.
- **NAT & Security:** NAT Gateway gives Fargate controlled outbound Internet access; security groups restrict traffic so only ALB → Fargate → RDS communication is allowed.

Prepare Network Infrastructure

The screenshot displays the AWS Management Console interface for a VPC named 'Grafana VPC' (ID: vpc-022a16d97e18fd465). The top navigation bar shows the AWS logo, a search bar, and the account ID (7976-6676-1058) in the United States (N. Virginia) region.

VPC Details:

- VPC ID:** vpc-022a16d97e18fd465
- State:** Available
- Tenancy:** default
- Default VPC:** No
- Network Address Usage metrics:** Disabled
- Block Public Access:** Off
- DHCP option set:** dopt-090f1a1ed755ca019
- IPv4 CIDR:** 10.0.0.0/16
- Route 53 Resolver DNS Firewall rule groups:** --
- DNS hostnames:** Disabled
- Main route table:** rts-0798f771931f62308 / Private Route Table
- IPv6 pool:** --
- Owner ID:** 797666781058

Resource map:

- VPC:** Grafana VPC
- Subnets (3):** Subnets within this VPC
 - us-east-1a: Public Subnet
 - us-east-1b: Private Subnet 1
 - us-east-1c: Private Subnet 2
- Route tables (2):** Route network traffic to resources
 - Private Route Table
 - Public Route Table
- Network Connections (1):** Connections to other networks
 - Grafana IGW

The resource map shows connections between the subnets and the route tables, and between the route tables and the Internet Gateway (IGW).

A VPC with public and private subnets was created and an Internet Gateway attached. Route tables and security groups for ALB, ECS tasks, and RDS were configured.

Create PostgreSQL RDS Instance

The screenshot displays the AWS Management Console interface for a PostgreSQL RDS instance named 'grafana-db'. The top navigation bar shows the AWS logo, a search bar, and the account ID '7976-6678-1058'. The breadcrumb trail indicates the path: Aurora and RDS > Databases > grafana-db. The instance's status is 'Available', and its class is 'db.t4g.micro'. The engine is 'PostgreSQL' and the region is 'us-east-1a'. The 'Connectivity & security' tab is selected, showing the endpoint 'grafana-db.cqzg0yssggd9.us-east-1.rds.amazonaws.com' on port '5432'. The networking details include the availability zone 'us-east-1a', VPC 'Grafana VPC (vpc-022a16d97e18fd465)', and subnet group 'default-vpc-022a16d97e18fd465'. The security settings show the instance is not publicly accessible and has a VPC security group 'default (sg-0776e10a149e7cc82)'.

grafana-db [Modify](#) [Actions](#)

Summary

DB identifier grafana-db	Status Available	Role Instance	Engine PostgreSQL	Recommendations
CPU -	Class db.t4g.micro	Current activity	Region & AZ us-east-1a	

[Connectivity & security](#) [Monitoring](#) [Logs & events](#) [Configuration](#) [Zero-ETL integrations](#) [Maintenance & backups](#) [Data migrations - new](#) [Tags](#) [Re](#)

Connectivity & security

Endpoint & port	Networking	Security
Endpoint grafana-db.cqzg0yssggd9.us-east-1.rds.amazonaws.com	Availability Zone us-east-1a	VPC security groups default (sg-0776e10a149e7cc82) Active
Port 5432	VPC Grafana VPC (vpc-022a16d97e18fd465)	Publicly accessible No
	Subnet group default-vpc-022a16d97e18fd465	Certificate authority info rds-ca-rsa2048-g1
	Subnets subnet-0a821683792f3fda6 subnet-04d825b95bd68a23c subnet-0aecdac097fecd08	Certificate authority date May 26, 2061, 06:34 (UTC+07:00)
		DB instance certificate expiration date September 23, 2026, 21:35 (UTC+07:00)

[CloudShell](#) [Feedback](#) © 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

A PostgreSQL RDS instance metabase-db was launched in private subnets. The endpoint and credentials were recorded.

Create ECS Cluster (Fargate)

Cluster configuration

Cluster name
grafana-ecs-cluster
Cluster name must be 1 to 255 characters. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

► **Service Connect defaults - optional**

▼ **Infrastructure - optional** [info](#) Serverless
Your cluster is automatically configured for AWS Fargate (serverless) with two capacity providers. Add Amazon EC2 instances.
☒ **AWS Fargate (serverless)**
Pay as you go. Use if you have tiny, batch, or burst workloads or for zero maintenance overhead. The cluster has Fargate and Fargate Spot capacity providers by default.
☐ **Amazon EC2 instances**
Manual configurations. Use for large workloads with consistent resource demands.
☐ **External instances using ECS Anywhere** can be registered after cluster creation is complete.

► **Monitoring - optional**
Configure observability, encryption, and logging options to maintain compliance and operational visibility of your container environment.

► **Encryption - optional**
Choose the KMS keys used by tasks running in this cluster to encrypt your storage.

► **Tags - optional** [info](#)
Tags help you to identify and organize your clusters.

[Cancel](#) [Create](#)

On the Create Cluster page in the ECS console, the cluster type Networking only (Fargate) was selected and a cluster named metabase-fargate-cluster was created with the default VPC and networking configuration.

Cluster Successfully Provisioned

The screenshot displays the AWS Management Console for the 'grafana-ecs-cluster'. The cluster is in an 'Active' state, as indicated by the green status icon and the 'Active' label in the 'Status' section. The 'Cluster overview' section shows the ARN, status, CloudWatch monitoring, and registered container instances. The 'Services' section shows a 'Draining' service with 1 active instance. The 'Tasks' section shows a 'Pending' task with 1 instance. The 'Infrastructure' tab is selected, showing 'Capacity providers (2)' and 'Container instances (0)'. The 'Capacity providers' table lists 'Fargate' and 'Fargate Spot' as available providers. The 'Container instances' section shows no instances are currently running.

Cluster overview

ARN: `arn:aws:ecs:us-east-1:797666781058:cluster/grafana-ecs-cluster`

Status: **Active**

CloudWatch monitoring: **Default**

Registered container instances: **0**

Services

Draining: **1** Active

Tasks

Pending: **1**

Running: **0**

Capacity providers (2)

Capacity provider	ASG	Manage...	Manage...	Manage...	Current ...	Desired ...	Min size	Max size	Update status
Fargate	-	-	-	-	-	-	-	-	-
Fargate Spot	-	-	-	-	-	-	-	-	-

Container instances (0)

Register external instances: **0**

Actions: **0**

The Cluster Details page now highlights the cluster in an Active state, showing that it's fully set up and ready to deploy Fargate services without any further configuration needed.

Create ECS Task Definition

The screenshot displays the AWS Management Console interface for creating a new ECS task definition. The page is titled "Create new task definition" and is divided into several sections:

- Task definition configuration:** This section includes fields for "Task definition family" (postgres-task) and "Task type" (Fargate).
- Infrastructure requirements:** This section includes fields for "Platform" (Linux) and "Architecture" (x86_64).
- Containers:** This section is expanded, showing the configuration for a container named "postgres". Key settings include:
 - Name:** postgres
 - Image URI:** public.ecr.aws/docker/library/postgres:13.3
 - Container role:** ecs-tasks-role
 - Task role:** ecs-task-role
 - CPU:** 512
 - Memory:** 2048
 - Environment variables:** POSTGRES_USER=postgres, POSTGRES_PASSWORD=postgres, POSTGRES_DB=postgres
- Monitoring:** This section is collapsed.

On the Create Task Definition page, a new task definition was created with 0.5 vCPU, 2GB memory, and a container using the postgres image for the PostgreSQL database connection.

The postgres database-task was set up with 0.5 vCPU, 2GB memory, and exposed to port 3000. Environment variables were set for the PostgreSQL database connection.

Task

The screenshot displays the AWS Management Console for the 'metabase-ecs-cluster'. The top navigation bar shows the AWS logo, a search bar, and the account ID '7976-6678-1058'. The breadcrumb trail indicates the path: Amazon Elastic Container Service > Clusters > metabase-ecs-cluster > Tasks.

Cluster overview

- ARN:** arn:aws:ecs:us-east-1:797666781058:cluster/metabase-ecs-cluster
- Status:** Active (indicated by a green checkmark)
- CloudWatch monitoring:** Default
- Registered container instances:** -

Services

- Draining:** -
- Active:** 1

Tasks

- Pending:** -
- Running:** 1

Tasks (1)

Filter tasks by property or value

Filter desired status: Any desired status | Filter launch type: Any launch type

☐	Task	Last status	Desired st...	Task definition	Health sta...	Created at	Started by	Started at	Grp
☐	0099b6e8073d4d04a980...	Running	Running	metabase-task:1	Unknown	12 minutes ago	ecs-svc/75346028070...	12 minutes ago	serv

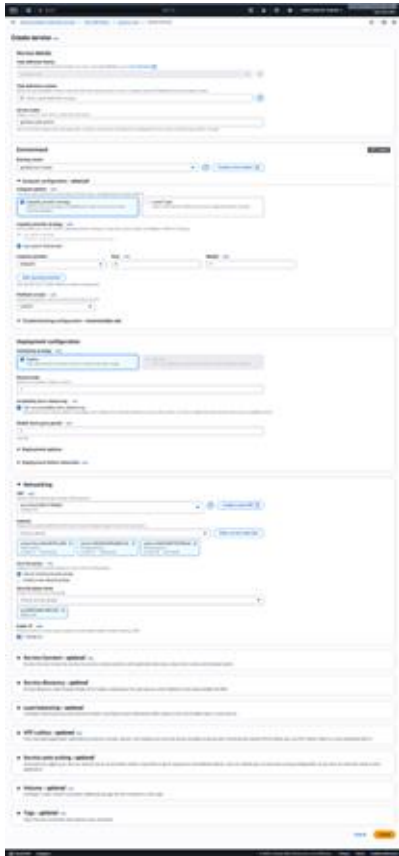
The Task Definition Details page confirms the new revision is ACTIVE, signaling that the container configuration and database connection settings were successfully registered and are ready for deployment

Task Definition Successfully Registered

The screenshot displays the AWS Management Console interface for a task definition named `metabase-task:1`. The **Overview** tab is selected, showing the task definition's status as **Active**. Key details include the task role, task execution role, operating system/architecture (Linux/ARM_64), and app environment (EC2). Below the overview, the **Task size** section provides visual representations of the task's CPU and memory requirements. The **Container: metabase** section is expanded, showing the image used, private registry status, memory hard/soft limits, and CPU allocation. The **Environment and secrets** tab is also visible, showing no environment variables or secrets are currently defined.

The task definition was successfully registered and is now available as an active revision, ready for deployment of the Metabase container.

Create and Deploy ECS Service with ALB



- On the Create Service page, an ECS service named metabase-service was configured to run one task in the private subnets and integrated with a newly created Application Load Balancer listening on port 80. The ECS task security group and target group for traffic forwarding were also attached.

Service Successfully Deployed

The screenshot displays the AWS Management Console for the 'metabase-ecs-cluster'. The top navigation bar shows the AWS logo, a search bar, and the account ID '2976-6678-1058'. The breadcrumb trail indicates the path: Amazon Elastic Container Service > Clusters > metabase-ecs-cluster > Services.

The 'metabase-ecs-cluster' overview section shows the cluster is 'Active' and has 1 task running. The 'Services' tab is selected, displaying a table with the following data:

Service name	ARN	Status	Schedu...	Lau...	Task de...	Deployments and tasks	Last deployment
metabase-task-service	arn:aws:ecs:us-east-1:797666781058:cluster/metabase-ecs-cluster	Active	REPLICA	-	metabase...	1/1 Tasks running	Completed View

The text overlay states: 'The Service Details page shows the service in a Running and Stable state, with the Application Load Balancer marked as Active, confirming that traffic is now seamlessly routed to the ECS tasks through the load balancer.'

At the bottom of the console, there is a footer with 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc. or its affiliates, along with links for 'Privacy', 'Terms', and 'Cookie preferences'.

Service

aws

Search

[Alt+S]

United States (N. Virginia)

Account ID: 7976-6678-1058

Gye Goon Woo

Amazon Elastic Container Service

Clusters

metabase-ecs-cluster

Tasks

194d9b1a61584a9a92ffa8a89c7cc70

Configuration

Configuration

Operating system/Architecture

Linux/X86_64

CPU | Memory

1 vCPU | 3 GB

Platform version

1.4.0

Task execution role

[ecsTaskExecutionRole](#)

Task role

-

Fault injection

Turned off

ECS Exec

Info

Turned off

Capacity provider

Fargate

Launch type

Fargate

Task definition: revision

[metabase-task-3](#)

Task group

service:metabase-task-service | [View service](#)

ENI ID

[eni-0b1589ea49c69bdd9](#)

Network mode

awsvpc

Subnet

[subnet-0465476090d89acd9](#)

Public IP

[13.223.94.111](#) | [open address](#)

Private IP

[10.0.0.142](#)

MAC address

[02:10:ff:8e:72:79](#)

Task scale-in protection

Configure task scale-in protection

Protection status

Turned off

Protection duration

-

Containers (1)

Filter containers

Connect

< 1 >

Container details for metabase

CloudShell

Feedback

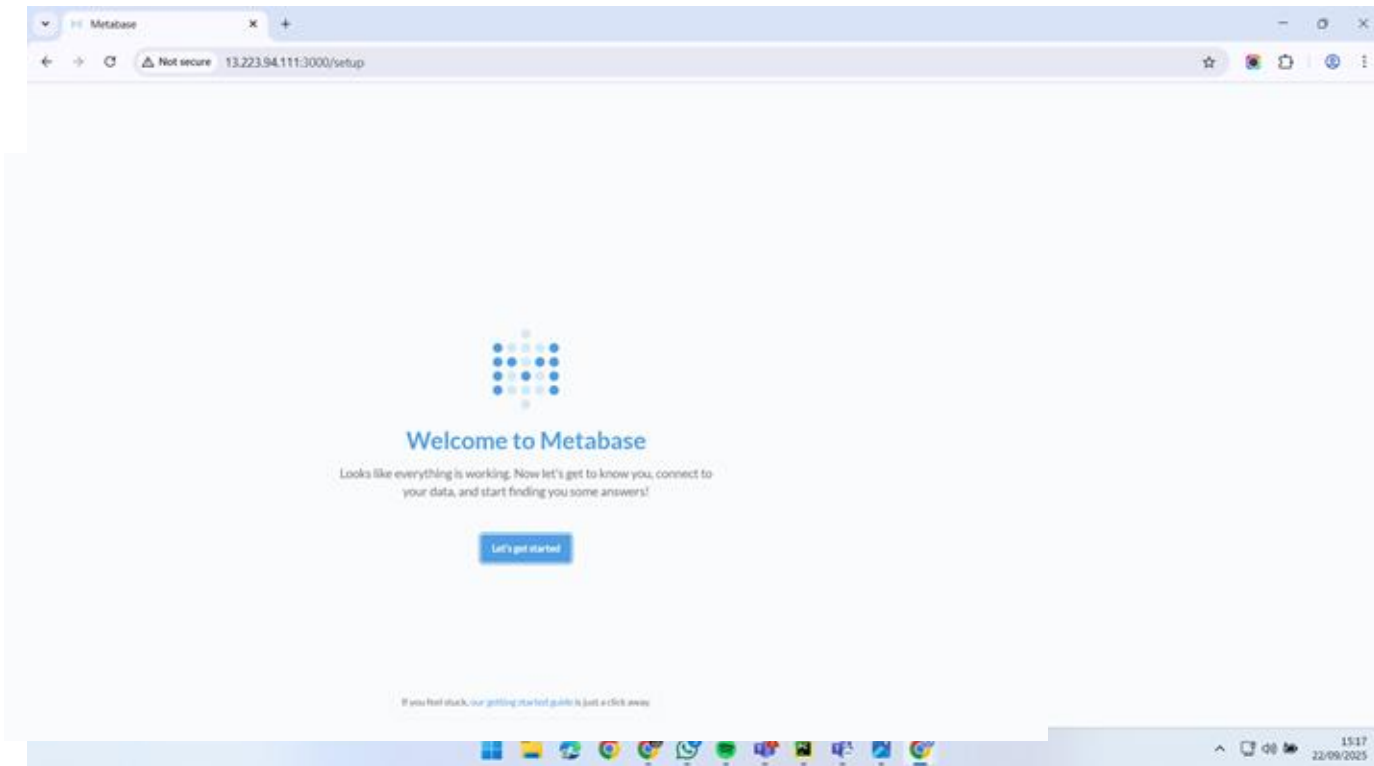
© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Access and Test Metabase



The Metabase application was accessed via the ALB DNS name and the setup wizard was completed. The connection to the PostgreSQL RDS backend was successfully established.

PostgreSQL Testing

The screenshot shows the Metabase Admin interface with a purple header bar. The header contains the Metabase logo, 'Metabase Admin', and navigation links: 'Settings', 'Databases', 'Table Metadata', 'People', 'Permissions', 'Performance', 'Tools', and 'Exit admin'. The 'Databases' link is highlighted.

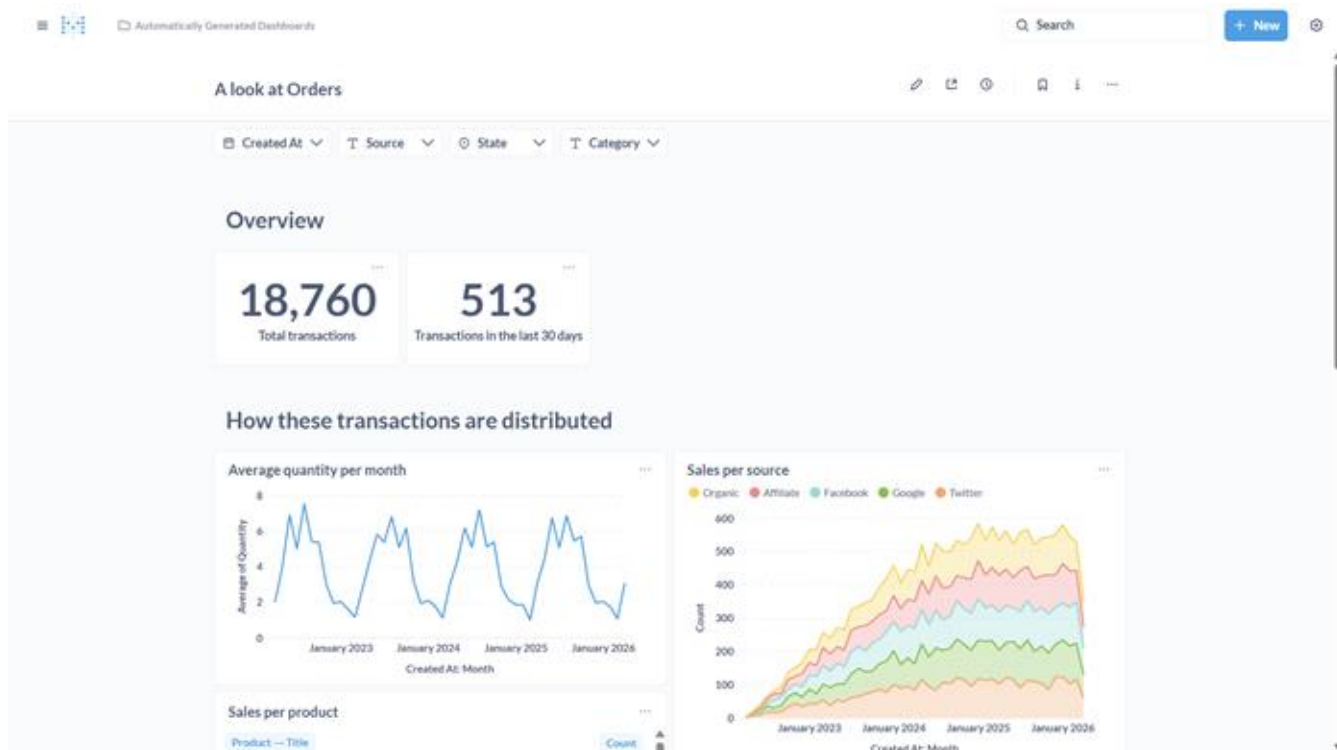
Below the header, the breadcrumb 'DATABASES > POSTGRESQL' is visible. The main heading is 'PostgreSQL', with a subtitle 'PostgreSQL Added September 21, 2025'.

The interface is divided into three main sections:

- Connection and sync:** This section includes a 'Connected' status indicator with a green dot and a link to 'Edit connection details'. Below this are two buttons: 'Sync database schema' and 'Re-scan field values'.
- Model features:** This section includes a 'Model actions' toggle switch, which is currently turned on. Below the toggle is a note: 'Allow actions from models created from this data to be run. Actions are able to read, write, and possibly delete data. Note: Your database user will need write permissions.'
- Danger zone:** This section includes two red buttons: 'Discard saved field values' and 'Remove this database'.

Options for Configuring Storage Size.

EC2 Instance Success Running



Confirmation of the successful launch of the EC2 instance, with details about the instance, such as instance ID, public and private IP addresses, and instance state.

Best Practices for Production Deployment

- For a production environment, it is strongly recommended to use an **Application Load Balancer (ALB)** to ensure high availability, automatic traffic distribution, and easier scaling of ECS tasks.
- ALB provides built-in health checks and better traffic routing, which improves reliability and performance compared to exposing ECS tasks directly.
- Although this setup was tested on a **free tier account** where ALB is **not fully supported**, following the ALB-based architecture remains the industry best practice for secure and scalable deployments.
- If free tier limitations are a concern, consider upgrading the AWS account or using a paid plan to leverage ALB features for long-term stability.

Key Takeaways

- A complete Metabase environment was successfully deployed on AWS using **ECS Fargate** and **PostgreSQL RDS**, with a network architecture designed for both scalability and security.
- The environment follows a production-ready design, separating public and private subnets and implementing security groups for controlled access.
- While the current free tier environment cannot fully implement ALB, the architecture has been built with ALB integration in mind, ensuring an easy upgrade path to a production-ready setup when moving to a paid plan.
- This approach provides a solid foundation to scale, secure, and manage Metabase in a cloud-native infrastructure.

Thank You